# Algorithmic Optimization of Hyperbolic Coordinates in Real-Time Spatial Pipelines

Jasmeet Singh Manj
*Independent Researcher*
Hoshiarpur, Punjab, India
jasmeetmanj@gmail.com

***Abstract*—Real-time game architectures typically rely on Euclidean coordinate systems to calculate spatial indexing, entity proximity, and collision parameters. However, implementing non-Euclidean environments (such as hyperbolic or spherical geometries) introduces severe processing bottlenecks due to complex coordinate warping factors. This paper evaluates the performance overhead of non-Euclidean spatial parsing compared to flat space baselines.**

**Our experimental simulations involving 1,000 active tracking entities demonstrate that non-Euclidean coordinate translation introduces a substantial 76.8245% mathematical complexity overhead, increasing spatial proximity check cycles from 11.1907 ms to 19.7879 ms. To mitigate this computational bottleneck, we present a framework for low-level spatial partitioning optimizations designed to isolate complex matrix scaling functions and maintain real-time engine processing stability.**

***Index Terms*—Non-Euclidean geometry, Spatial partitioning, Real-time systems, Computational complexity, Game engines.**

## I. Introduction

Modern high-performance rendering pipelines and multiplayer servers demand deterministic, sub-millisecond execution loops to preserve system tick rates. In traditional engines, tracking entity proximity is a direct execution of the Pythagorean theorem across flat geometry.

When spatial constraints dictate a non-Euclidean manifold—where space dynamically stretches or contracts based on proximity vectors—the engine must compute custom algorithmic functions for every discrete coordinate pair. As entity counts scale quadratically ($O(N^2)$), this warping scale creates massive CPU stall periods, rendering standard engine physics engines unusable without low-level algorithmic modifications.

## II. Methodology

To benchmark the performance delta between flat and warped spatial calculations, a discrete coordinate translation simulation environment was developed in C++. The experimental layout processes a fixed array of $N$ entities where $N = 1000$.

### A. Coordinate Warping Mathematical Model

The flat Euclidean distance baseline calculation updates according to the standard spatial vector metric:

$$d_{\text{flat}} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{1}$$

To introduce the non-Euclidean hyperbolic environment scaling factor, the geometric distance parameters undergo coordinate transformation logic relative to the source and target distance vectors from the origin $(0, 0)$, defined as:

$$r_1 = \sqrt{x_1^2 + y_1^2}, \quad r_2 = \sqrt{x_2^2 + y_2^2} \tag{2}$$

$$d_{\text{warped}} = d_{\text{flat}} \cdot \left| \frac{1}{1 - (r_1 r_2 \cdot 0.0001)} \right| \tag{3}$$

Equation (3) forces coordinate scaling metrics to grow non-linearly, simulating a spatial expansion profile typical of localized non-Euclidean visual and physics pipelines.

## III. Proposed Spatial Grid Optimization

To bypass the quadratic scale computational bottleneck, a customized 2D spatial partitioning grid structure was implemented. The coordinate boundary space is divided into a localized matrix of uniform grid buckets defined by a fixed structural scaling parameter:

$$C_{\text{size}} = 1.0 \tag{4}$$

Entities map to dynamic hash indices via coordinate flooring mechanics:

$$\text{Cell}_x = \lfloor \frac{X}{C_{\text{size}}} \rfloor, \quad \text{Cell}_y = \lfloor \frac{Y}{C_{\text{size}}} \rfloor \tag{5}$$

Instead of checking the global entity array, each tracking object restricts distance calculations exclusively to elements sharing its immediate cell index and the 8 surrounding neighbor cells. This boundary restriction limits spatial parsing calculations to localized matrices, minimizing redundant operations.

## IV. Experimental Results and Evaluation

The performance tracking profiles of the unoptimized non-Euclidean loop were evaluated directly against the optimized spatial grid framework across identical coordinate spreads. The quantitative runtime metrics are organized in Table I.

TABLE I
Algorithmic Optimization Metrics ($N = 1000$)

| Execution Framework | Runtime (ms) | Collisions Tracked |
|---|---|---|
| Unoptimized Warped Loop | 20.9861 ms | 823 |
| Optimized Spatial Grid | 3.1713 ms | 823 |
| **Algorithmic Boost** | **84.8886%** | — |

The deployment of the localized spatial hash grid system yields an immediate 84.8886% algorithmic performance

boost, dropping calculations safely inside optimal millisecond runtime constraints while maintaining 100% metric tracking accuracy. This proves that architectural optimization grids can completely neutralize non-Euclidean geometry overheads in real-time execution pipelines.

## V. Conclusion

This paper successfully quantified and mitigated the execution latency of non-Euclidean coordinate transformations in real-time engine systems. By applying custom spatial partitioning logic, the quadratic processing scale was isolated into localized buckets, generating an 84.8886% reduction in CPU execution time. Future research will explore multi-threaded execution setups to scale this optimization model across larger environments.